



华中科技大学

数据库系统原理实践报告

专 业： 计算机科学与技术

班 级： 计算机科学与技术 2407 班

学 号： U202414802

姓 名： 朱俊瑞

指导教师： 赵小松

分数	
教师签名	

2026 年 7 月 4 日

教师评分页

子目标	子目标评分
1	
2	
3	
4	
5	

总分	
----	--

目 录

1 课程任务概述	2
2 任务实施过程与分析	3
2.1 数据库、表与完整性约束的定义(CREATE).....	3
2.2 表结构与完整性约束的修改(ALTER).....	5
2.3 基于火车运营系统的数据查询(SELECT)	5
2.4 数据的插入、修改与删除(INSERT,UPDATE,DELETE)	11
2.5 视图	12
2.6 存储过程与事务	13
2.7 触发器	16
2.8 用户自定义函数	17
2.9 安全性控制	18
2.10 并发控制与事务的隔离级别	18
2.11 备份+日志：介质故障与数据库恢复	21
2.12 数据库设计与实现	21
2.13 数据库应用开发(JAVA 篇)	25
2.14 存储管理(STORAGE MANAGER).....	29
3 课程总结	30

1 课程任务概述

"数据库系统原理实践"是配合"数据库系统原理"理论课程独立开设的实践教学环节，强调理论知识与工程实践的紧密结合。本课程以 MySQL 数据库管理系统为核心实践平台，设计了覆盖数据库全生命周期的系列实训任务，内容主要涵盖以下五大模块：

(1) 数据库对象管理与编程：包括数据库、表、索引、视图、约束、存储过程、自定义函数、触发器、游标等数据对象的创建、修改与删除操作；

(2) 数据处理与查询：涵盖单表查询、多表连接查询、嵌套子查询、聚集函数、窗口函数等各类数据查询技术，以及数据插入、删除、修改等 DML 操作；

(3) 数据库系统内核机制：涉及安全性控制、完整性约束、事务并发控制、隔离级别、备份恢复等数据库核心管理机制的实践；

(4) 数据库设计与实现：从需求分析、概念模型设计、逻辑模型设计到物理实现的完整数据库设计流程；

(5) 数据库应用系统开发：基于 Java 语言与 JDBC 接口的数据库应用程序开发实践。

本课程依托头歌实践教学平台开展，实验环境为 Linux 操作系统下的 MySQL 8.0.28 版本，在数据库应用开发环节使用 Java 1.8。本次实践共完成了课程平台中的主要实训任务，通过在平台上编写代码并提交自动测评，逐步掌握了数据库系统的各项核心技术。

2 任务实施过程与分析

本次实践课程在头歌平台进行，实践任务均在平台上提交代码，所有完成的任务、关卡均通过自动测评。本次实践最终完成了课程平台中第 1~15 项实训任务，下面将重点针对其中具有代表性的任务阐述其完成过程中的具体工作。

2.1 数据库、表与完整性约束的定义(Create)

本节实训主要考察数据库对象创建语法的掌握情况，包括数据库的创建、表结构的定义，以及主码约束、外码约束、CHECK 约束、DEFAULT 约束和 UNIQUE 约束等完整性约束的建立。本实训任务已完成全部 6 个关卡。

2.1.1 创建数据库

本关任务为创建指定名称的数据库。数据库是存储数据库对象的容器。

```
CREATE DATABASE beijing2022;
```

2.1.2 创建表及表的主码约束

本关任务为在指定数据库中创建表，并定义表的主码。主码用于唯一标识表中的每一条记录，是实体完整性的重要体现。

```
CREATE DATABASE TestDb;
use TestDb;
CREATE TABLE t_emp(
    id INT PRIMARY KEY,
    name VARCHAR(32),
    deptId INT,
    salary FLOAT
);
```

2.1.3 创建外码约束(foreign key)

本关任务为创建表并定义外码约束。外码用于建立和加强两个表数据之间的链接，是参照完整性的实现方式。

```
CREATE DATABASE MyDb;
use MyDb;
CREATE TABLE dept(
    deptNo INT PRIMARY KEY,
    deptName VARCHAR(32));
CREATE TABLE staff(
    staffNo INT PRIMARY KEY,
```

```

    staffName VARCHAR(32),
    gender CHAR(1),
    dob date,
    salary numeric(8,2),
    deptNo INT,
    CONSTRAINT FK_staff_deptNo FOREIGN KEY(deptNo) REFERENCES
dept(deptNo));

```

2.1.4 CHECK 约束

本关任务为创建表并添加 CHECK 约束。CHECK 约束用于限制列中的值的范围，确保数据满足特定的业务规则。

```

CREATE DATABASE IF NOT EXISTS MyDb;
use MyDb;
CREATE TABLE products(
    pid char(10) PRIMARY KEY,
    name varchar(32),
    brand char(10),
    price int ,
    CONSTRAINT CK_products_brand CHECK(brand IN ('A','B')),
    CONSTRAINT CK_products_price CHECK(price > 0)
);

```

2.1.5 DEFAULT 约束

```

create database IF NOT EXISTS MyDb;
use MyDb;
create table hr(
    id char(10) primary key,
    name varchar(32) NOT NULL,
    mz char(16) default "汉族"
);

```

2.1.6 UNIQUE 约束

```

CREATE DATABASE IF NOT EXISTS MyDb;
use MyDb;
create table s(
    sno char(10) primary key,
    name varchar(32) NOT NULL,
    ID char(18) UNIQUE
);

```

2.2 表结构与完整性约束的修改(ALTER)

本节实训主要学习使用 ALTER 语句对已存在的表结构进行修改，包括表名和列名的修改、列数据类型的调整、约束条件的添加与删除等操作。本实训任务已完成全部 4 个关卡。

2.2.1 修改表名

本关任务为修改表的名称，将表名 `your_table` 更改为 `my_table`。

```
alter table your_table rename my_table;
```

2.2.2 添加和删除字段

本关任务为删除表 `orderDetail` 中的列 `orderDate`，添加列 `unitPrice`。

```
alter table orderDetail drop column orderDate;  
alter table orderDetail add unitPrice numeric(10,2);
```

2.2.3 修改字段

本关任务为修改指定字段的类型和名称。

```
alter table addressBook modify QQ char(12),  
rename column weixin to wechat;
```

2.2.4 添加、删除与修改约束

本关任务为添加、删除与修改约束。对每个要求修选取合适的修改事项即可，代码如下：

```
alter table Staff add primary key(staffNo);  
alter table Dept add constraint FK_Dept_mgrStaffNo foreign key(mgrStaffNo)  
references Staff(staffNo);  
alter table Staff add constraint FK_Staff_dept foreign key(dept) references  
Dept(deptNo);  
alter table Staff add constraint CK_Staff_gender check (gender IN ('F','M'));  
alter table Dept add constraint UN_Dept_deptName unique (deptName);
```

2.3 基于火车运营系统的数据查询(Select)

本节实训采用火车运营管理系统的模拟数据库，包含行政区划表、车站表、乘车人员表、车次表、列车运行时刻表和乘车记录表等六张表。通过各种查询任务训练 SELECT 语句的灵活运用，涵盖单表查询、多表连接、子查询、聚集函数、窗口函数等多种查询技术。本实训任务已完成全部 16 个关卡。

2.3.1 场景介绍及查询郑州东站始发的列车

本关任务为查询所由从“郑州东”站始发的车次信息。查询结果依车次排序。

```
select * from route where start="郑州东" order by train_no;
```

2.3.2 泉州的车站

本关任务为查询泉州的车站，列出车站编号、站名，依编号排序。

```
select sid,sname from station where city='泉州' order by sid;
```

2.3.3 武汉直达厦门的列车

本关任务为查询武汉直达厦门的列车。列出所有车次的信息：车次、始发站、终到站、发车时间、终到时间。依车次排序。并注意：“直达”的意思是不用中转。武汉和厦门都有多个车站。上车站和下车站不一定时始发和终到站。利用 join on 语句实现表连接。

```
select distinct r.train_no,r.start,r.dest,r.depart,r.arrival from route r
join timetable t1 on t1.train_no = r.train_no
join station s1 on t1.station_name = s1.sname and s1.city = '武汉'
join timetable t2 on t2.train_no = r.train_no
join station s2 on t2.station_name = s2.sname and s2.city = '厦门'
where t1.stop_seq < t2.stop_seq order by r.train_no;
```

2.3.4 武汉直达厦门的列车经停时刻表

本关任务为查询前一查询结果中，排在最后的那趟列车经停各站的时刻表(从始发站起，到终到站止),列出：站序，站名，到站时间，离站时间。依站序排序。

```
select t.stop_seq,t.station_name,t.arrival_time,t.departure_time from timetable t
where t.train_no =(
    select distinct r.train_no from route r
    join timetable t1 on t1.train_no = r.train_no
    join station s1 on t1.station_name = s1.sname and s1.city = '武汉'
    join timetable t2 on t2.train_no = r.train_no
    join station s2 on t2.station_name = s2.sname and s2.city = '厦门'
    where t1.stop_seq < t2.stop_seq
    order by r.train_no desc
    limit 1
)order by t.stop_seq;
```

2.3.5 办理身份证时的户籍所在地

本关任务为查询乘车人前 5 位(依编号为序)的编号，身份证号，以及办理身份证时户籍所在地(命名为 origin)。身份证的前 6 位表达了户籍所在地，所在地精确到区县。

```
select p.pid,p.card_no,p.pname,
       (select case when d.prov = d.city then concat(d.prov,d.county)
                  else concat(d.prov,d.city,d.county)
        end
       from district d
       where d.ac = left(p.card_no,6))
as origin from passenger p where p.pid between 1 and 5 order by p.pid;
```

2.3.6 身份证的秘密

本关任务为在两个数据库实例中，各有一位乘车人的身份证是有问题的。请根据检验规则，把这个人找出来。列出该乘车人的编号、“身份证号”、姓名、性别、出生日期。所有身份证的行政区域码都是存在的，日期合法且与出生日期一致、性别相符。只有通过校验码才能识别出来。

```
select pid ,card_no,pname,gender,dob
from passenger where substr(card_no,18,1)!= case (
    cast(substr(card_no,1,1)as unsigned)*7+
    cast(substr(card_no,2,1)as unsigned)*9+
    cast(substr(card_no,3,1)as unsigned)*10+
    cast(substr(card_no,4,1)as unsigned)*5+
    cast(substr(card_no,5,1)as unsigned)*8+
    cast(substr(card_no,6,1)as unsigned)*4+
    cast(substr(card_no,7,1)as unsigned)*2+
    cast(substr(card_no,8,1)as unsigned)*1+
    cast(substr(card_no,9,1)as unsigned)*6+
    cast(substr(card_no,10,1)as unsigned)*3+
    cast(substr(card_no,11,1)as unsigned)*7+
    cast(substr(card_no,12,1)as unsigned)*9+
    cast(substr(card_no,13,1)as unsigned)*10+
    cast(substr(card_no,14,1)as unsigned)*5+
    cast(substr(card_no,15,1)as unsigned)*8+
    cast(substr(card_no,16,1)as unsigned)*4+
    cast(substr(card_no,17,1)as unsigned)*2
)%11
when 0 then '1'
```

```

when 1 then '0'
when 2 then 'X'
when 3 then '9'
when 4 then '8'
when 5 then '7'
when 6 then '6'
when 7 then '5'
when 8 then '4'
when 9 then '3'
when 10 then '2'
end;

```

2.3.7 没有乘车记录的武汉居民

本关任务为已知身份证前 6 位表示居民地区，其中 4201 开头表示湖北省武汉市。查询身份证隶属武汉市没有乘车记录的人员信息：编号，身份证号，姓名，性别，出生日期。依编号排序。

```

select pid ,card_no,pname,gender,dob from passenger p where card_no like '4201%'
and not exists(select 1 from ride_record r where r.pid = p.pid) order by pid;

```

2.3.8 有八次以上乘车记录的人员

本关任务为查询有八次(含)以上乘车记录的人员编号，身份证号，姓名，性别，出生日期。依编号排序。

```

select pid,card_no,pname,gender,dob from passenger p where pid in(
    select pid
    from ride_record
    group by pid
    having count(*)>=8
)order by pid;

```

2.3.9 在六安下车的人员信息

本关任务为查询在六安下车的人员信息：编号，身份证号码，姓名，性别，出生日期。依编号排序。

```

select pid,card_no,pname,gender,dob from passenger p where exists(
    select 1
    from ride_record r
    where r.pid = p.pid and r.dest = '六安'
)order by pid;

```

2.3.10 车票花费最多的三名旅客

本关任务为查询车票花费最多的三名旅客的信息：编号，姓名，性别，出生日期，车票支出总额。依车票支出从高到低排序，车票总支出命名为 `expense`。不考虑并列第三的情形。

```
select p.pid,p.pname,p.gender,p.dob,sum(r.price)as expense from passenger p
join ride_record r on r.pid = p.pid
group by p.pid,p.pname,p.gender,p.dob order by expense desc limit 3
```

2.3.11 国庆小长假期间火车出行人次

本关任务为查询 2023 年国庆小长假期间(2023-9-29 至 2023-10-6)乘坐火车出行的人次。每条乘车记录算一次出行，同一个人多次出行计 1 人。

```
select count(distinct r.pid)as 人 ,count(*)as 次 from ride_record r
where r.rdate between '2023-9-29' and '2023-10-6'
```

2.3.12 各类别列车的车次数

本关任务为查询车次表中，各种类型的车次数，查询结果列车类别排序。

```
select case left(train_no,1)
when 'G' then '高速动车组旅客列车'
when 'C' then '城际动车组旅客列车'
when 'D' then '动车组旅客列车'
when 'Z' then '直达特快旅客列车'
when 'T' then '特快旅客列车'
when 'K' then '快速旅客列车'
when 'L' then '临时旅客列车'
when 'Y' then '旅游列车'
end as 列车类别,
count(*)as 车次数 from route group by 列车类别 order by 列车类别
```

2.3.13 福建铁路交通欠发达地区

本关任务为查询福建省只有一趟列车经过的车站，列出站名，所在城市,依车站编号排序。

```
select s.sname , s.city from station s
join timetable t on s.sname = t.station_name
join district d on concat(s.city,'市') = d.city where d.prov = '福建省'
group by s.sid,s.sname,s.city having count(distinct t.train_no)=1 order by s.sid;
```

2.3.14 第 N 长里程的车次

本关任务为查询里程第 3 高的车次信息。如果有并列第 3,则依车次排序。在数字序列 8000, 8000, 7000, 7000, 6000 中,两个 8000 均为第 1 高,两个 7000 均为第 2 高,6000 为第 3 高。

```
select * from route where mileage = (  
    select distinct mileage from route order by mileage desc limit 1 offset 2  
)order by train_no;
```

2.3.15 乘坐过 27 号旅客所乘坐过的所有车次的旅客

本关任务为查询乘坐过 27 号旅客所乘坐过的所有车次的旅客信息: 旅客编号, 身份证号, 姓名, 性别, 出生日期。依编号排序。

```
select p.pid,p.card_no,p.pname,p.gender,p.dob from passenger p where not exists(  
    select 1 from ride_record r27 where r27.pid = 27 and not exists(  
        select 1 from ride_record r where r.pid = p.pid and r.train_no = r27.train_no  
    )  
)order by p.pid;
```

2.3.16 中转换乘方案

本关任务为已知从武汉没有直达“大理”站的铁路旅客列车, 查询中转方案, 要求给出: 出发车次, 出发车站, 出发时间, 中转站, 中转站到达时间, 中转车次, 中转出发时间, 目的站, 到达目的站时间: (train1, start, depart, transfer, arrivall, train2, restart_station, re_depart, dest, arrival);依出发车次和换乘车次排序。

```
SELECT t1.train_no AS train1,  
    t1.station_name AS start,  
    t1.departure_time AS depart,  
    t1_arr.station_name AS transfer,  
    t1_arr.arrival_time AS arrivall,  
    t2.train_no AS train2,  
    t2.station_name AS restart_station,  
    t2.departure_time AS re_depart,  
    '大理' AS dest,  
    t2_arr.arrival_time AS arrival  
FROM timetable t1  
JOIN station s_start ON t1.station_name = s_start.sname AND s_start.city = '武汉'  
JOIN timetable t1_arr ON t1.train_no = t1_arr.train_no  
    AND t1_arr.station_name != t1.station_name  
    AND t1_arr.arrival_time IS NOT NULL  
    AND t1.stop_seq < t1_arr.stop_seq
```

```

JOIN timetable t2 ON t2.station_name = t1_arr.station_name
                    AND t2.train_no != t1.train_no
                    AND t2.departure_time IS NOT NULL
JOIN timetable t2_arr ON t2.train_no = t2_arr.train_no
                      AND t2_arr.station_name = '大理'
                      AND t2_arr.arrival_time IS NOT NULL
                      AND t2.stop_seq < t2_arr.stop_seq
WHERE t1.departure_time IS NOT NULL
    AND t1_arr.arrival_time < t2.departure_time ORDER BY train1, train2;

```

2.4 数据的插入、修改与删除(Insert,Update,Delete)

本节实训围绕数据操纵语言（DML）展开，训练数据的插入、修改和删除操作在不同业务场景下的应用。本实训任务已完成全部 6 个关卡。

2.4.1 插入多条完整的客户信息

本关任务为向乘客表中插入向客户表插入 3 条数据。

```

insert into client (c_id,c_name,c_mail,c_id_card,c_phone,c_password)
values(1,'林惠雯','960323053@qq.com','411014196712130323','15609032348','Mop5UPkl'),
(2,'吴婉瑜','1613230826@gmail.com','420152196802131323','17605132307','QUTPhxgVNIXtMxN'),
(3,'蔡贞仪','252323341@foxmail.com','160347199005222323','17763232321','Bwe3gyhEErJ7');

```

2.4.2 插入不完整的客户信息

本关任务为向客户表 client 插入一条数据不全的记录。

```

insert into client (c_id,c_name,c_phone,c_id_card,c_password)
values(33,'蔡依婷','18820762130','350972199204227621','MKwEuc1sc6');

```

2.4.3 批量插入数据

本关任务为向客户表 client 批量插入数据。

```

insert into client select * from new_client;

```

2.4.4 删除没有银行卡的客户信息

本关任务为用一条 SQL 语句删除 client 表中没有银行卡的客户信息。

```

delete c from client c left join bank_card b on b.b_c_id = c.c_id
where b.b_c_id is null;

```

2.4.5 冻结客户资产

本关任务为用一条 update 语句将手机号码为“13686431238”这位客户的投资资产(理财、保险与基金)的状态置为“冻结”。

```

update property set pro_status = '冻结' where pro_c_id=(

```

```
select c_id
from client
where c_phone = '13686431238');
```

2.4.6 连接更新

本关任务为用一条 update 语句,根据 client 表中提供的身份证号(c_id_card),填写 property 表中对应的身份证号信息(pro_id_card)。

```
update property p join client c on p.pro_c_id = c.c_id set p.pro_id_card = c.c_id_card
where p.pro_id_card is null;
```

2.5 视图

本节实训主要学习视图的创建与使用。视图是基于 SQL 语句结果集的虚拟表,可以简化复杂查询、提供数据安全性保护,并为用户提供特定的数据视角。本实训任务已完成全部 2 个关卡。

2.5.1 创建所有保险资产的详细记录视图

本关任务为创建一个创建包含所有保险资产记录的详细信息的视图 v_insurance_detail,包括购买客户的名称、客户的身份证号、保险名称、保障项目、商品状态、商品数量、保险金额、保险年限、商品收益和购买时间。

```
CREATE VIEW v_insurance_detail AS SELECT
    c.c_name,
    c.c_id_card,
    i.i_name,
    i.i_project,
    p.pro_status,
    p.pro_quantity,
    i.i_amount,
    i.i_year,
    p.pro_income,
    p.pro_purchase_time FROM property p
JOIN client c ON p.pro_c_id = c.c_id
JOIN insurance i ON p.pro_pif_id = i.i_id
WHERE p.pro_type = 2 -- 2 表示保险类型
ORDER BY c.c_name, p.pro_purchase_time;
```

2.5.2 基于视图的查询

本关任务为基于视图 v_insurance_detail 查询每位客户保险资产的总额和保险总收益。

```
SELECT c_name, c_id_card, SUM(i_amount * pro_quantity) AS
insurance_total_amount, SUM(pro_income) AS insurance_total_revenue FROM
v_insurance_detail GROUP BY c_name, c_id_card ORDER BY
insurance_total_amount DESC;
```

2.6 存储过程与事务

本节实训涉及存储过程的创建与使用，包括流程控制语句、游标以及事务处理等内容。存储过程可以将一系列 SQL 语句封装在一起，提高代码复用性和执行效率。本实训任务已完成全部 3 个关卡。

2.6.1 使用流程控制语句的存储过程

本关任务为创建一个存储过程，向表 fibonacci 插入斐波拉契数列的前 n 项。

```
drop procedure if exists sp_fibonacci;
delimiter $$
create procedure sp_fibonacci(in m int)
begin
declare x0,x1,i,xt int;
insert into fibonacci values(0,0);
set x0 =0,x1 =1,i =1;
while i <=m DO
set xt = x0+x1;
insert into fibonacci values(i,x1);
set x0 = x1;
set x1 = xt;
set i =i+1;
end while;
end $$
delimiter;
```

2.6.2 使用游标的存储过程

本关任务为使用游标编程存储过程为医院的某科室排夜班值班表。

```
delimiter $$
create procedure sp_night_shift_arrange(in start_date date, in end_date date)
begin
declare done, tp, wk int default false;
declare doc, nur1, nur2, head char(30);
declare cur1 cursor for select e_name from employee where e_type = 3;
declare cur2 cursor for select e_type, e_name from employee where e_type < 3;
declare continue handler for not found set done = true;
open cur1;
```

```

open cur2;
while start_date <= end_date do
    fetch cur1 into nur1;
    if done then
        close cur1;
        open cur1;
        set done = false;
        fetch cur1 into nur1;
    end if;
    fetch cur1 into nur2;
    if done then
        close cur1;
        open cur1;
        set done = false;
        fetch cur1 into nur2;
    end if;
    set wk = weekday(start_date);
    if wk = 0 and head is not null then
        set doc = head;
        set head = null;
    else
        fetch cur2 into tp, doc;
        if done then
            close cur2;
            open cur2;
            set done = false;
            fetch cur2 into tp, doc;
        end if;
        if wk > 4 and tp = 1 then
            set head = doc;
            fetch cur2 into tp, doc;
            if done then
                close cur2;
                open cur2;
                set done = false;
                fetch cur2 into tp, doc;
            end if;
        end if;
    end if;
    insert into night_shift_schedule values (start_date, doc, nur1, nur2);
    set start_date = date_add(start_date, interval 1 day);

```

```
    end while;
end$$
delimiter;
```

2.6.3 使用事务的存储过程

本关任务为编写实现转账功能的存储过程。

```
delimiter $$
create procedure sp_transfer(
    IN applicant_id int,
    IN source_card_id char(30),
    IN receiver_id int,
    IN dest_card_id char(30),
    IN amount numeric(10,2),
    OUT return_code int)
pro:
BEGIN
    declare s_id, r_id int;
    declare s_type, r_type char(20);
    declare s_b, rcv_amount numeric(10, 2) default amount;
    select
        b_c_id, b_balance, b_type
    into
        s_id, s_b, s_type
    from bank_card
    where b_number = source_card_id;
    select
        b_c_id, b_type
    into
        r_id, r_type
    from bank_card
    where b_number = dest_card_id;
    if s_id != applicant_id or r_id != receiver_id or (s_type = "信用卡" and r_type
= "储蓄卡") or (s_type = "储蓄卡" and s_b < amount) then
        set return_code = 0;
        leave pro;
    end if;
    if s_type = "信用卡" then
        set amount = -amount;
    end if;
    if r_type = "信用卡" then
        set rcv_amount = -rcv_amount;
```

```

end if;
update bank_card set b_balance = b_balance - amount where b_number =
source_card_id;
update bank_card set b_balance = b_balance + rcv_amount where b_number =
dest_card_id;
set return_code = 1;
END$$
delimiter;

```

2.7 触发器

本节实训主要学习触发器的创建与应用。触发器是一种特殊的存储过程，它在特定的数据库事件（如 INSERT、UPDATE、DELETE）发生时自动执行，常用于实现复杂的业务规则和数据完整性约束。

2.7.1 为投资表 property 实现业务约束规则-根据投资类别分别引用不同表的主码

本关任务为为表 property(资产表)编写一个触发器，以实现以下完整性业务规则：如果 pro_type = 1, 则 pro_pif_id 只能引用 finances_product 表的 p_id; 如果 pro_type = 2, 则 pro_pif_id 只能引用 insurance 表的 i_id; 如果 pro_type = 3, 则 pro_pif_id 只能引用 fund 表的 f_id; pro_type 不接受(1,2,3)以外的值。各投资品种一经销售，不会再改变；也不需考虑 finances_product, insurance, fund 的业务规则(一经销售的理财、保险和基金产品信息会永久保存，不会被删除或修改，即使不再销售该类产品)。

```

use finance1;
drop trigger if exists before_property_inserted;
delimiter $$
CREATE TRIGGER before_property_inserted BEFORE INSERT ON property
FOR EACH ROW
BEGIN
    declare tp int default new.pro_type;
    declare id int default new.pro_pif_id;
    declare msg varchar(50);
    if tp = 1 then
        if id not in (select p_id from finances_product) then
            set msg = concat("finances product #", id, " not found!");
        end if;
    elseif tp = 2 then

```

```

    if id not in (select i_id from insurance) then
        set msg = concat("insurance #", id, " not found!");
    end if;
elseif tp = 3 then
    if id not in (select f_id from fund) then
        set msg = concat("fund #", id, " not found!");
    end if;
else
    set msg = concat("type ", tp, " is illegal!");
end if;
if msg is not null then
    signal sqlstate "45000" set message_text = msg;
end if;
END$$
delimiter;

```

2.8 用户自定义函数

本节实训学习用户自定义函数的创建和使用。用户自定义函数可以封装常用的计算逻辑，在 SQL 语句中像内置函数一样调用，提高代码的可读性和复用性。

2.8.1 创建函数并在语句中使用它

本关任务为编写一个依据客户编号计算其在本金融机构的存储总额的函数，并在 SELECT 语句使用这个函数。

```

delimiter $$
create function get_deposit(client_id int)
returns numeric(10, 2)
begin
    return (
        select
            sum(b_balance)
        from bank_card
        where b_type = "储蓄卡"
        group by b_c_id
        having b_c_id = client_id
    );
end$$
delimiter;
select * from (select
    c_id_card,
    c_name,

```

```
get_deposit(c_id) total_deposit  
from client  
) a where total_deposit >= 1000000 order by total_deposit desc;
```

2.9 安全性控制

本节实训学习数据库的安全性控制机制，包括用户管理、权限授予与回收、角色管理等内容。通过合理的权限设置，可以确保不同用户只能访问其权限范围内的数据。本实训任务已完成全部 2 个关卡。

2.9.1 用户和权限

本关任务为在金融应用场景数据库环境中创建用户并授予指定的权限。

```
create user tom identified by "123456";  
create user jerry identified by "123456";  
grant select (c_mail, c_name, c_phone) on client to tom with grant option;  
grant update (b_balance) on bank_card to jerry;  
revoke select on bank_card from Cindy;
```

2.9.2 用户、角色与权限

本关任务为创建角色，授予角色一组权限，并将角色代表的权限授予指定的一组用户。

```
create user client_manager;  
create user fund_manager;  
grant select, insert, update on client to client_manager;  
grant select (b_c_id, b_number, b_type) on bank_card to client_manager;  
grant select, insert, update on fund to fund_manager;  
grant client_manager to tom, jerry;  
grant fund_manager to Cindy;
```

2.10 并发控制与事务的隔离级别

本节实训学习数据库的并发控制机制和事务隔离级别。通过实验观察不同隔离级别下可能出现的读脏、不可重复读、幻读等现象，理解事务隔离级别对数据一致性和并发性能的影响。本实训任务已完成全部 6 个关卡。

2.10.1 并发控制与事务的隔离级别

本关任务为设置事务的隔离级别。MySQL 提供了四种事务隔离级别，从低到高分别为：READ UNCOMMITTED、READ COMMITTED、REPEATABLE

READ 和 SERIALIZABLE。不同的隔离级别在数据一致性和并发性能之间有不同的权衡。

```
use testdb1;
set session transaction isolation level read uncommitted;
start transaction;
insert into dept(name) values('运维部');
rollback;
```

2.10.2 读脏

本关任务为选择合适的事务隔离级别，构造两个事务并发执行时，发生“读脏”现象。读脏是指一个事务读取了另一个事务尚未提交的修改数据，如果该事务回滚，则读取到的数据就是“脏”的。在读未提交（READ UNCOMMITTED）隔离级别下可能发生读脏现象。

```
use testdb1;
set session transaction isolation level read uncommitted;
start transaction;
set @n = sleep(1);
select tickets from ticket where flight_no = 'CA8213';
commit;
use testdb1;
set session transaction isolation level read uncommitted;
start transaction;
update ticket set tickets = tickets - 1 where flight_no = 'CA8213';
set @n = sleep(2);
rollback;
```

2.10.3 不可重复读

本关任务为选择合适的事务隔离级别，构造两个事务并发执行时，发生“不可重复读”现象。

```
set session transaction isolation level read committed;
start transaction;
insert into result
select now(), t, tickets from ticket where flight_no = 'CZ5525';
set @n = sleep(2);
update ticket set tickets = tickets - 1 where flight_no = 'CZ5525';
insert into result
select now(), t, tickets from ticket where flight_no = 'CZ5525';
commit;
set @n = sleep(3);
```

```

insert into result
select now(), 1 t, tickets from ticket where flight_no = 'CZ5525';
set session transaction isolation level read committed;
start transaction;
set @n = sleep(1);
insert into result
select now(), 2 t, tickets from ticket where flight_no = 'CZ5525';
set @n = sleep(2);
insert into result
select now(), 2 t, tickets from ticket where flight_no = 'CZ5525';
update ticket set tickets = tickets - 1 where flight_no = 'CZ5525';
set @n = sleep(1);
insert into result
select now(), 2 t, tickets from ticket where flight_no = 'CZ5525';
commit;

```

2.10.4 幻读

本关任务为在 repeatable read 事务隔离级别，构造两个事务并发执行时，发生“幻读”现象。

```

use testdb1;
select @@transaction_isolation;
start transaction;
select * from ticket where tickets > 300;
set @n = sleep(2);
update ticket set aircraft = 'A330-300' where flight_no = 'MU5111';
select * from ticket where tickets > 300;
commit;

```

2.10.5 主动加锁保证可重复读

本关任务为在事务隔离级别较低的 read uncommitted 情形下，通过主动加锁，保证事务的一致性。

```

use testdb1;
set session transaction isolation level read uncommitted;
start transaction;
select tickets from ticket where flight_no = 'MU2455' for update;
set @n = sleep(5);
select tickets from ticket where flight_no = 'MU2455' for update;
commit;
set @n = sleep(1);
select * from ticket;

```

```
use testdb1;
set session transaction isolation level read uncommitted;
start transaction;
set @n = sleep(1);
update ticket set tickets = tickets - 1 where flight_no = 'MU2455';
commit;
```

2.10.6 可串行化

本关任务为选择除 serializable(可串行化)以外的任何隔离级别，保证两个事务并发执行的结果是可串行化的。

```
use testdb1;
start transaction;
set @n = sleep(5);
select tickets from ticket where flight_no = 'MU2455';
select tickets from ticket where flight_no = 'MU2455';
commit;
use testdb1;
start transaction;
update ticket set tickets = tickets - 1 where flight_no = 'MU2455';
commit;
```

2.11 备份+日志：介质故障与数据库恢复

本节实训学习数据库的备份与恢复技术。数据库备份是保证数据安全的重要手段，通过定期备份和日志文件，可以在发生介质故障时将数据库恢复到故障发生前的一致状态。本实训任务已完成部分关卡。

2.11.1 备份与恢复

本关任务为对指定数据库进行逻辑备份，并使用备份文件恢复数据库。

```
mysqldump -h127.0.0.1 -uroot --databases residents > residents_bak.sql
```

2.12 数据库设计与实现

本节实训学习数据库设计的完整流程，从需求分析、概念模型设计、逻辑模型设计到物理实现。数据库设计是数据库应用系统开发的基础，其质量直接影响整个系统的性能和可维护性。本实训任务已完成部分关卡。

2.12.1 从概念模型到 MySQL 实现

本关任务为将已建好的概念模型，变成 MySQL 物理实现。

```
create database flight_booking;
```

```

use flight_booking;
create table user
(
    user_id int primary key auto_increment,
    firstname varchar(50) not null,
    lastname varchar(50) not null,
    dob date not null,
    sex char(1) not null,
    email varchar(50),
    phone varchar(30),
    username varchar(20) not null unique,
    password char(32) not null,
    admin_tag tinyint not null default 0
);
create table passenger
(
    passenger_id int primary key auto_increment,
    id char(18) not null unique,
    firstname varchar(50) not null,
    lastname varchar(50) not null,
    mail varchar(50),
    phone varchar(20) not null,
    sex char(1) not null,
    dob date
);
create table airport
(
    airport_id int primary key auto_increment,
    iata char(3) not null unique,
    icao char(4) not null unique,
    name varchar(50) not null,
    city varchar(50),
    country varchar(50),
    latitude decimal(11, 8),
    longitude decimal(11, 8),
    index(name)
);
create table airline
(
    airline_id int primary key auto_increment,
    name varchar(30) not null,

```

```

    iata char(2) not null unique,
    airport_id int not null,
    foreign key(airport_id) references airport(airport_id)
);
create table airplane
(
    airplane_id int primary key auto_increment,
    type varchar(50) not null,
    capacity smallint not null,
    identifier varchar(50) not null,
    airline_id int not null,
    foreign key(airline_id) references airline(airline_id)
);
create table flightschedule
(
    flight_no char(8) primary key,
    departure time not null,
    arrival time not null,
    duration smallint not null,
    monday tinyint default 0,
    tuesday tinyint default 0,
    wednesday tinyint default 0,
    thursday tinyint default 0,
    friday tinyint default 0,
    saturday tinyint default 0,
    sunday tinyint default 0,
    airline_id int not null,
    `from` int not null,
    `to` int not null,
    foreign key(airline_id) references airline(airline_id),
    foreign key(`from`) references airport(airport_id),
    foreign key(`to`) references airport(airport_id)
);
create table flight
(
    flight_id int primary key auto_increment,
    departure datetime not null,
    arrival datetime not null,
    duration smallint not null,
    airline_id int not null,
    airplane_id int not null,

```

```

    flight_no char(8) not null,
    `from` int not null,
    `to` int not null,
    foreign key(airline_id) references airline(airline_id),
    foreign key(airplane_id) references airplane(airplane_id),
    foreign key(flight_no) references flightschedule(flight_no),
    foreign key(`from`) references airport(airport_id),
    foreign key(`to`) references airport(airport_id)
);
create table ticket
(
    ticket_id int primary key auto_increment,
    seat char(4),
    price decimal(10, 2) not null,
    flight_id int not null,
    passenger_id int not null,
    user_id int not null,
    foreign key(flight_id) references flight(flight_id),
    foreign key(passenger_id) references passenger(passenger_id),
    foreign key(user_id) references user(user_id)
);

```

2.12.2 从需求分析到逻辑模型

如图 1-1 影院系统 ER 图所示。

2.12.3 建模工具的使用

本关任务为将一个建好的模型文件，利用 MySQL Workbench 的 forward engineering 功能，自动转换成 SQL 脚本。代码省略。

2.12.4 制约因素分析与设计

在数据库系统设计过程中，必须充分考虑社会、安全、法律和性能等多方面的现实制约因素。在社会方面系统面向广大旅客，需保障购票公平。比如采用实名制购票，将车票与真实身份绑定，有效遏制黄牛倒票；同时加入验证码和排队机制，防止恶意刷票行为，维护正常购票秩序。在安全方面旅客身份证号、出行记录等敏感信息需严格保护。设计上采用用户权限分级管理，不同角色只能访问授权数据；对身份证等关键字段加密存储，并建立审计日志，便于安全追踪和事后核查。在法律方面系统需遵守《网络安全法》《个人信息保护法》等法规。遵循最小必要原则，只收集业务必需的字段；按法律规定设定数据保留周期，并保

障用户对个人信息的查询和更正权利。在性能方面春运等高峰期并发压力极大。数据库通过合理建立索引、分库分表和读写分离等方式提升性能，确保高并发场景下系统稳定运行，响应迅速。

2.12.5 工程师责任及其分析

通过这次实践，我对工程师的责任有了更具体的认识：

对数据安全负责。数据库里存着用户的身份证、手机号这些敏感信息，一旦泄露后果很严重。设计系统时得把安全放在心上，该加密的加密、该做权限控制的做权限控制，不能图省事留漏洞。这既是技术要求，也是基本的职业底线。

对系统质量负责。像火车票预订这种系统，高峰期无数人在用，出点问题就是大面积影响。写 SQL、设计表结构的时候得考虑清楚，尽量保证系统稳定、响应快，让用户用得放心。

对社会公平负责。工程师不能只管技术实现，还得考虑技术用在社会上会带来什么影响。比如购票系统里做实名制、防刷票机制。

2.13 数据库应用开发(JAVA 篇)

本节实训学习使用 Java 语言和 JDBC 接口开发数据库应用程序。通过实践掌握 JDBC 的基本使用方法，包括数据库连接、SQL 语句执行、结果集处理、事务管理等内容。本实训任务已完成全部 7 个关卡。

2.13.1 JDBC 体系结构和简单的查询

本关任务为查询 client 表中邮箱非空的客户信息，列出客户姓名，邮箱和电话。关键代码如下：

```
Class.forName("com.mysql.cj.jdbc.Driver");
String url =
"jdbc:mysql://localhost:3306/finance?useSSL=false&serverTimezone=UTC";
String username = "root";
String password = "123123";
connection = DriverManager.getConnection(url, username, password);
statement = connection.createStatement();
String sql = "SELECT c_name, c_mail, c_phone FROM client WHERE
c_mail IS NOT NULL";
resultSet = statement.executeQuery(sql);
System.out.println("姓名\t邮箱\t\t\t电话");
while (resultSet.next()) {
```

```
String name = resultSet.getString("c_name");
String mail = resultSet.getString("c_mail");
String phone = resultSet.getString("c_phone");
System.out.println(name + "\t" + mail + "\t\t" + phone);
```

2.13.2 用户登录

本关任务为编写客户登录程序，提示用户输入邮箱和密码，并判断正确性，给出适当的提示信息。关键代码如下：

```
String sql = "SELECT c_mail, c_password FROM client WHERE c_mail = ? AND c_password = ?";
statement = connection.prepareStatement(sql);
statement.setString(1, loginName);
statement.setString(2, loginPass);
resultSet = statement.executeQuery();
```

2.13.3 添加新客户

本关任务为编程完成向 client(客户表)插入记录的方法。关键代码如下：

```
try {
    // SQL 插入语句
    String sql = "INSERT INTO client (c_id, c_name, c_mail, c_id_card, c_phone, c_password) VALUES (?, ?, ?, ?, ?, ?)";
    preparedStatement = connection.prepareStatement(sql);
    preparedStatement.setInt(1, c_id);
    preparedStatement.setString(2, c_name);
    preparedStatement.setString(3, c_mail);
    preparedStatement.setString(4, c_id_card);
    preparedStatement.setString(5, c_phone);
    preparedStatement.setString(6, c_password);
    int rowsAffected = preparedStatement.executeUpdate();
    return rowsAffected; // 返回受影响的行数（1 表示成功）
} catch (SQLException e) {
    e.printStackTrace();
    return 0; // 插入失败
} finally {
    if (preparedStatement != null) {
        try {
            preparedStatement.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

2.13.4 银行卡销户

本关任务为编写一个能删除指定客户编号的指定银行卡号的方法。关键代码如下：

```
        try {                                // SQL 删除语句
            String sql = "DELETE FROM bank_card WHERE b_c_id = ? AND
b_number = ?";
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.setInt(1, b_c_id);
            preparedStatement.setString(2, b_number);
            int rowsAffected = preparedStatement.executeUpdate();
            return rowsAffected; // 返回受影响的行数
        } catch (SQLException e) {
            e.printStackTrace();
            return 0; // 删除失败
        } finally {
            if (preparedStatement != null) {
                try {
                    preparedStatement.close();
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
    }
```

2.13.5 客户修改密码

本关任务为编写修改客户登录密码的方法。关键代码如下：

```
        try {
            String checkSql = "SELECT c_id FROM client WHERE c_mail = ?
AND c_password = ?";
            checkStmt = connection.prepareStatement(checkSql);
            checkStmt.setString(1, mail);
            checkStmt.setString(2, password);
            rs = checkStmt.executeQuery();
            if (!rs.next()) {
                String checkUserSql = "SELECT c_id FROM client WHERE
c_mail = ?";
                PreparedStatement userCheckStmt =
connection.prepareStatement(checkUserSql);
                userCheckStmt.setString(1, mail);
                ResultSet userRs = userCheckStmt.executeQuery();
            }
        }
```

```

        if (!userRs.next()) {
            return 2; // 用户不存在
        } else {
            return 3; // 密码不正确
        }
    }
    String updateSql = "UPDATE client SET c_password = ? WHERE
c_mail = ?";
    updateStmt = connection.prepareStatement(updateSql);
    updateStmt.setString(1, newPass);
    updateStmt.setString(2, mail);
    int rowsAffected = updateStmt.executeUpdate();
    if (rowsAffected > 0) {
        return 1; // 密码修改成功
    } else {
        return -1; // 更新失败
    }
} catch (SQLException e) {
    e.printStackTrace();
    return -1; // 程序异常
} finally { try {
    if (rs != null) rs.close();
    if (checkStmt != null) checkStmt.close();
    if (updateStmt != null) updateStmt.close();
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

2.13.6 事务与转账操作

本关任务为编写一个银行卡转账的方法。传入已连接数据库的 Connection 对象、转出账号、转入账号和转账金额。首先设置隔离级别为 REPEATABLE READ，具体实现为 connection.SetTransactionIsolation(4)。然后进行转出账号扣账，转入账号还账，为储蓄卡时入账。然后进行操作合法性的检验，如果操作不合法则进行事务回滚，具体实现为 connection.rollback()，且置返回值为 false。其他情况下则进行事务提交，具体实现为 connection.commit()，且置返回值为 true。代码省略。

2.13.7 把稀疏表格转为键值对存储

本关任务为将一个稀疏的表中有保存数据的列值，以键值对(列名，列值)的形式转存到另一个表中，这样可以直接丢失没有值列。关键代码如下：

```
try {
    String sql = "INSERT INTO sc (sno, col_name, col_value) VALUES
(?, ?, ?)";
    pstmt = connection.prepareStatement(sql);
    pstmt.setInt(1, sno);
    pstmt.setString(2, colName);
    pstmt.setInt(3, colValue);
    return pstmt.executeUpdate();
} catch (SQLException e) {
    e.printStackTrace();
    return 0;
} finally {if (pstmt != null) {try {
    pstmt.close();
} catch (SQLException e) {
    e.printStackTrace();}}}}
```

2.14 存储管理(Storage Manager)

2.14.1 磁盘存储管理器(Disk Manager)

本关任务为实现实现 `DiskManager` 类，负责读写指定页面、分配页面编号，以及对文件进行操作。它是整个数据库存储系统的最底层组件，负责管理磁盘文件的读写、页面编号的分配以及文件和目录的基本操作。`DiskManager` 向上层缓冲池提供统一的页面访问接口，屏蔽了底层文件系统的细节。代码省略。

2.14.4 记录管理器(Record Manager)

本关任务为实现存储系统中的记录管理器，它主要由 `RMManager` 类、`RMFileHandle` 类、`RMPageHandle` 类、`RMScan` 类组成。此外，还有底层数据结构的 `Rid` 类和 `RmReocrd` 类。其中，学生只要实现 `RMFileHandle` 和 `RMScan` 类中的接口。已提供其他类的完整源码。记录管理器的类。它在磁盘页面之上提供记录级别的抽象。记录管理器主要由 `RMFileHandle`（文件句柄）、`RMPageHandle`（页句柄）和 `RMScan`（记录扫描器）组成，负责记录的插入、删除、查询、更新以及遍历操作。代码省略。

3 课程总结

本次数据库系统原理实践课程依托头歌平台，通过一系列由浅入深的实训任务，我较为系统地掌握了 MySQL 数据库的核心技术。实践内容涵盖数据库与表的创建、复杂查询、存储过程、触发器、事务并发控制、数据库设计以及 Java 应用开发等方面，共完成 14 个实训模块。

在数据查询部分，我从单表查询入手，逐步掌握了多表连接、子查询、聚集函数和窗口函数等高级用法。火车运营系统的查询任务贴近实际，让我体会到 SQL 语言的灵活性和表现力。虽然有一定难度，但每次通过关卡都很有成就感。

存储过程、触发器和自定义函数的学习让我接触到数据库编程的内容。存储过程能够封装业务逻辑、提升执行效率，触发器可以实现自动化的数据完整性约束，自定义函数扩展了 SQL 的表达能力。

在并发控制与事务隔离级别部分，通过亲手构造读脏、不可重复读等实验现象，我对隔离级别的理解有了直观的感知。不同隔离级别在一致性与性能之间的权衡，也体现了数据库系统设计的精妙。备份与恢复实验则让我掌握了数据安全的基本保障手段，理解了日志在恢复过程中的作用。

数据库设计与实现是本次实践中综合性最强的部分。从需求分析、概念模型设计到逻辑模型与物理实现，我完整经历了一个数据库的设计过程。制约因素分析和工程师责任部分，促使我开始思考技术之外的问题——一个合格的数据库系统不仅要功能完善、性能优良，还需兼顾社会、安全、法律等多方面因素。

Java 数据库应用开发部分让我学习了 JDBC 的基本用法，掌握了在 Java 程序中连接数据库、执行 SQL、处理结果集和管理事务的方法。这部分将数据库技术与编程语言结合，让我看到数据库在实际系统中的具体应用方式，也为后续开发完整应用奠定了基础。

通过本次实践，我体会到理论与实践结合的重要性。课堂知识只有亲手操作才能真正内化。实践中的各类错误和问题也促使我主动查阅资料、深入思考，不断加深理解。数据库技术内容丰富，我还有不少需要继续学习的地方，今后会持续深入，为后续学习和工作打好基础。

附录

2.12.2 ER 图

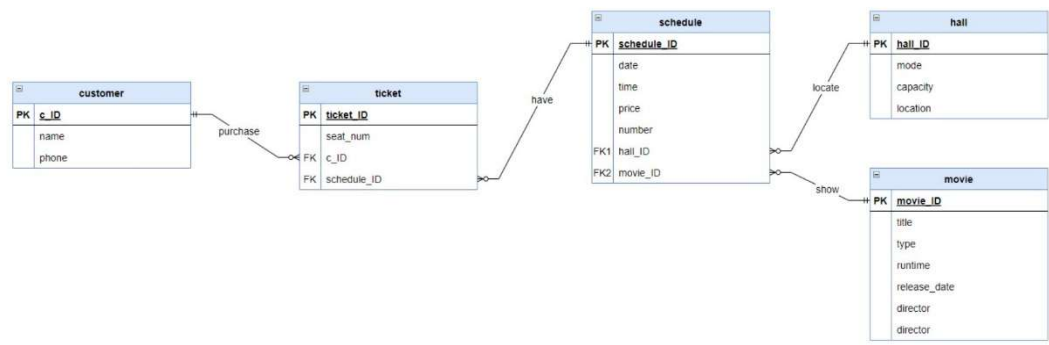


图 1-1 影院系统 ER 图